



Universidade Federal
de Campina Grande



UNIVERSIDADE FEDERAL DE CAMPINA GRANDE
DEPARTAMENTO DE ENGENHARIA ELÉTRICA E INFORMÁTICA
PROGRAMA DE RESIDÊNCIA EM MICROELETRÔNICA
MCTI/Softex/IRede

RELATÓRIO TÉCNICO

AES e Protocolo de comunicação SPI

Aluno: Guilherme José Mendes Jerônimo

Professor: Prof. Henrique Cunha

Semestre: 2026.2

Campina Grande - PB

2026

Sumário

1	Introdução	1
	Parte I – AES (FIPS 197)	1
2	Visão geral do AES	1
3	Notação e o Estado (<i>State</i>)	1
4	Matemática básica: o corpo $GF(2^8)$	2
5	A cifra: estrutura geral	2
6	As quatro transformações	3
6.1	SubBytes() – substituição não linear	3
6.2	ShiftRows() – deslocamento de linhas	4
6.3	MixColumns() – mistura das colunas	4
6.4	AddRoundKey() – soma com a chave de rodada	5
6.5	Exemplo numérico: rodada 1 do AES-128	5
7	Expansão de chave	5
8	Cifra inversa	6
9	Vetores de teste e implementação	7
	Parte II – Protocolo SPI	7
10	Visão geral do SPI	7
11	Sinais e conexão	8
12	Transmissão de dados	8
13	Polaridade e fase do clock (CPOL e CPHA)	8
14	Configuração com vários escravos	9
14.1	Modo regular	9

14.2 Modo <i>daisy-chain</i>	10
15 Estudo de caso: chaves e muxes com interface SPI	10
16 Resumo comparativo	11
Referências	12

1 Introdução

Este relatório resume dois temas importantes em sistemas embarcados e segurança da informação:

- o **AES** (*Advanced Encryption Standard*), cifra de bloco simétrica especificada no FIPS 197 do NIST;
- o **SPI** (*Serial Peripheral Interface*), interface serial síncrona muito usada entre microcontroladores e circuitos periféricos.

Os resumos seguem a ordem dos documentos originais, de forma mais direta, com figuras ilustrativas desenhadas em TikZ.

Parte I – AES (FIPS 197)

2 Visão geral do AES

O AES especifica o algoritmo **Rijndael**, uma cifra de bloco simétrica: a mesma chave secreta cifra (*encipher*) e decifra (*decipher*) os dados. O texto claro (*plaintext*) vira texto cifrado (*ciphertext*) e vice-versa.

- Blocos de dados de **128 bits** (16 bytes).
- Chaves de **128, 192 ou 256 bits** (AES-128, AES-192 e AES-256).
- Publicado em 26/11/2001 (FIPS 197); em 09/05/2023 foi atualizado como *FIPS 197-upd1*, com melhorias editoriais e **sem mudanças técnicas** no algoritmo.
- Pode ser implementado em software, firmware, hardware ou combinação; deve ser usado com um modo de operação aprovado.
- Não foram identificadas chaves fracas ou semifracas: não há restrição na escolha da chave.

Tabela 1: Combinações chave–bloco–rodadas (Figura 4 do FIPS 197).

	Chave (N_k palavras)	Bloco (N_b palavras)	Rodadas (N_r)
AES-128	4 (128 bits)	4 (128 bits)	10
AES-192	6 (192 bits)	4 (128 bits)	12
AES-256	8 (256 bits)	4 (128 bits)	14

3 Notação e o Estado (*State*)

Todas as operações são feitas sobre o **Estado**, uma matriz de bytes com 4 linhas e $N_b = 4$ colunas. Cada coluna é uma *palavra* de 32 bits. Os 16 bytes de entrada são copiados **coluna a coluna**:

$$s[r, c] = in[r + 4c], \quad 0 \leq r < 4, 0 \leq c < N_b. \quad (1)$$

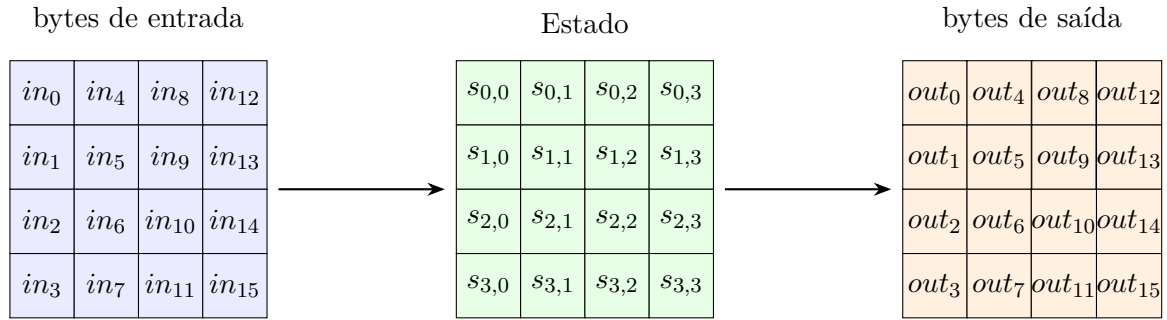


Figura 1: Entrada $\rightarrow$ Estado $\rightarrow$ saída (Figura 3 do FIPS 197).

Um byte é escrito em hexadecimal entre chaves (por exemplo, $\{63\} = \{01100011\}$) e interpretado como um polinômio em $GF(2^8)$: $b_7x^7 + b_6x^6 + \dots + b_1x + b_0$.

4 Matemática básica: o corpo $GF(2^8)$

Adição é o XOR bit a bit (e a subtração é igual à adição). Exemplo: $\{57\} \oplus \{83\} = \{d4\}$.

Multiplicação (denotada $\bullet$) é a multiplicação de polinômios módulo o polinômio irreduzível

$$m(x) = x^8 + x^4 + x^3 + x + 1 \quad (\{01\}\{1b\}). \quad (2)$$

Exemplo: $\{57\} \bullet \{83\} = \{c1\}$.

Multiplicação por x ($\{02\}$), chamada `xtime()`: desloca 1 bit à esquerda e, se o bit mais significativo era 1, faz XOR com $\{1b\}$. Qualquer multiplicação por constante sai de aplicações repetidas de `xtime()` somadas:

$$\begin{aligned} \{57\} \bullet \{02\} &= \{ae\}, & \{57\} \bullet \{04\} &= \{47\}, & \{57\} \bullet \{08\} &= \{8e\}, & \{57\} \bullet \{10\} &= \{07\} \\ \{57\} \bullet \{13\} &= \{57\} \oplus \{ae\} \oplus \{07\} = \{fe\}. \end{aligned}$$

Polinômios de 4 termos (colunas do Estado) são multiplicados módulo $x^4 + 1$, de modo que $x^i \bmod (x^4 + 1) = x^{i \bmod 4}$. O AES usa o polinômio fixo invertível

$$a(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\}, \quad a^{-1}(x) = \{0b\}x^3 + \{0d\}x^2 + \{09\}x + \{0e\}. \quad (3)$$

5 A cifra: estrutura geral

Após uma adição inicial de chave de rodada, o Estado passa por N_r rodadas. Cada rodada tem quatro transformações; a **última rodada não tem MixColumns()**.

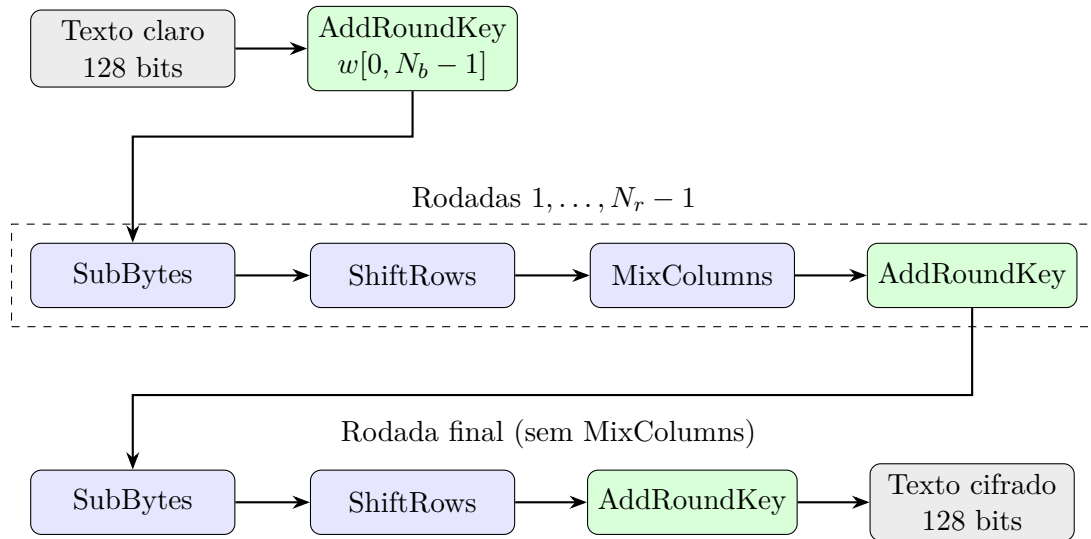


Figura 2: Fluxo da cifra AES (pseudocódigo da Figura 5 do FIPS 197).

```

Cipher(byte in[4*Nb], byte out[4*Nb], word w[Nb*(Nr+1)])
begin
  byte state[4,Nb]
  state = in
  AddRoundKey(state, w[0, Nb-1])
  for round = 1 step 1 to Nr-1
    SubBytes(state)
    ShiftRows(state)
    MixColumns(state)
    AddRoundKey(state, w[round*Nb, (round+1)*Nb-1])
  end for
  SubBytes(state)
  ShiftRows(state)
  AddRoundKey(state, w[Nr*Nb, (Nr+1)*Nb-1])
  out = state
end

```

6 As quatro transformações

6.1 SubBytes() – substituição não linear

Cada byte do Estado é substituído de forma independente usando a **S-box**, construída em dois passos: (1) inverso multiplicativo em $GF(2^8)$ (com $\{00\} \mapsto \{00\}$) e (2) transformação afim sobre $GF(2)$:

$$b'_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i, \quad c = \{63\}. \quad (4)$$

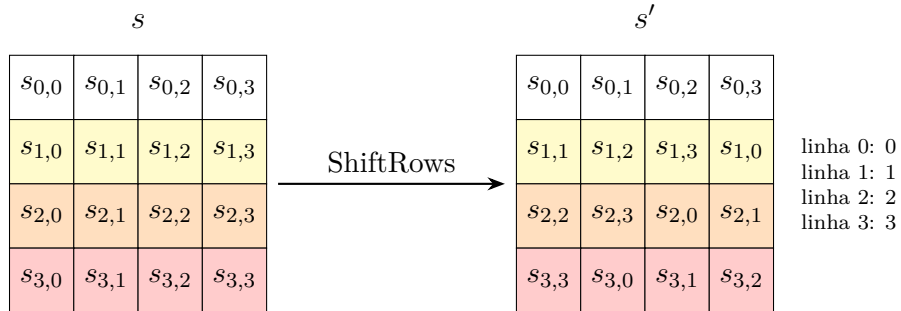
Exemplo: se $s_{1,1} = \{53\}$, olha-se a linha 5 e a coluna 3 da S-box e obtém-se $s'_{1,1} = \{ed\}$.

Tabela 2: S-box: valor de substituição para o byte xy (linha x , coluna y).

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

6.2 ShiftRows() – deslocamento de linhas

A linha 0 não muda; as linhas 1, 2 e 3 são deslocadas ciclicamente para a esquerda por 1, 2 e 3 bytes: $s'_{r,c} = s_{r,(c+\text{shift}(r,N_b)) \bmod N_b}$, com $\text{shift}(1,4) = 1$, $\text{shift}(2,4) = 2$, $\text{shift}(3,4) = 3$.

**Figura 3:** ShiftRows() desloca ciclicamente as três últimas linhas (Figura 8 do FIPS 197).

6.3 MixColumns() – mistura das colunas

Cada coluna é tratada como polinômio e multiplicada por $a(x)$ módulo $x^4 + 1$, o que equivale à multiplicação matricial em $\text{GF}(2^8)$:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}, \quad 0 \leq c < N_b. \quad (5)$$

Por exemplo: $s'_{0,c} = (\{02\} \bullet s_{0,c}) \oplus (\{03\} \bullet s_{1,c}) \oplus s_{2,c} \oplus s_{3,c}$. Esta é a operação que difunde cada byte por toda a coluna.

6.4 AddRoundKey() – soma com a chave de rodada

Cada coluna do Estado sofre XOR com uma palavra da chave expandida:

$$[s'_{0,c}, s'_{1,c}, s'_{2,c}, s'_{3,c}] = [s_{0,c}, s_{1,c}, s_{2,c}, s_{3,c}] \oplus [w_{round \cdot N_b + c}], \quad 0 \leq round \leq N_r. \quad (6)$$

Como é apenas XOR, essa transformação é a sua própria inversa. É a única etapa que usa a chave.

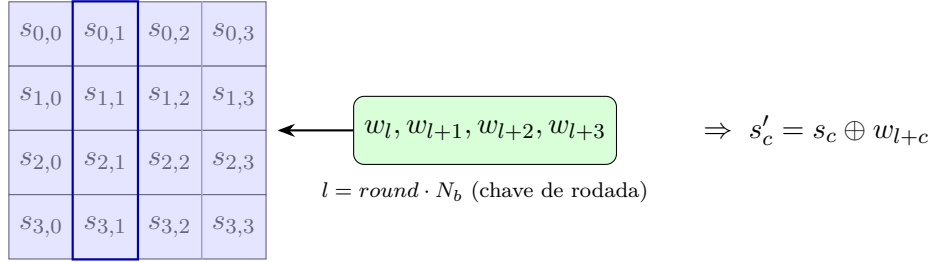


Figura 4: AddRoundKey(): cada coluna do Estado recebe XOR com uma palavra da chave.

6.5 Exemplo numérico: rodada 1 do AES-128

O Apêndice B do padrão acompanha o Estado em cada etapa. Os valores abaixo vêm da entrada e da chave:

- Entrada: 3243f6a8885a308d313198a2e0370734
- Chave: 2b7e151628aed2a6abf7158809cf4f3c

Na rodada 1:

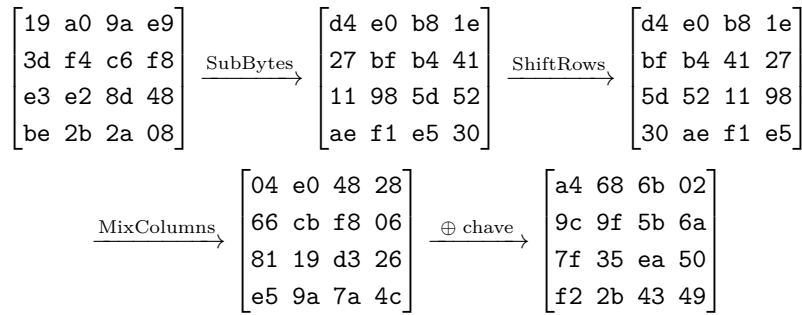


Figura 5: Rodada 1 do exemplo do Apêndice B (a última matriz é o início da rodada 2). Chave de rodada 1: colunas a0fafe17, 88542cb1, 23a33939, 2a6c7605.

7 Expansão de chave

A chave K gera $N_b(N_r + 1)$ palavras w_i (44 para AES-128, 52 para AES-192, 60 para AES-256):

- as primeiras N_k palavras são a própria chave;
- cada $w[i] = w[i - 1] \oplus w[i - N_k]$;

- quando i é múltiplo de N_k , antes do XOR aplica-se em $w[i - 1]$: `RotWord()` (rotação de bytes), `SubWord()` (S-box nos 4 bytes) e XOR com a constante de rodada `Rcon[i/Nk]`;
- no AES-256 ($N_k = 8$), se $i \bmod N_k = 4$, aplica-se apenas `SubWord()`.

```

KeyExpansion(byte key[4*Nk], word w[Nb*(Nr+1)], Nk)
begin
  i = 0
  while (i < Nk)
    w[i] = word(key[4*i], key[4*i+1], key[4*i+2], key[4*i+3])
    i = i+1
  end while
  i = Nk
  while (i < Nb * (Nr+1))
    temp = w[i-1]
    if (i mod Nk = 0)
      temp = SubWord(RotWord(temp)) xor Rcon[i/Nk]
    else if (Nk > 6 and i mod Nk = 4)
      temp = SubWord(temp)
    end if
    w[i] = w[i-Nk] xor temp
    i = i + 1
  end while
end

```

Tabela 3: Início da expansão da chave 2b7e1516 28aed2a6 abf71588 09cf4f3c (AES-128, Apêndice A.1).

i	temp	RotWord	SubWord	Rcon	XOR Rcon	$w[i - N_k]$	$w[i]$
4	09cf4f3c	cf4f3c09	8a84eb01	01000000	8b84eb01	2b7e1516	a0fafe17
5	a0fafe17					28aed2a6	88542cb1
6	88542cb1					abf71588	23a33939
7	23a33939					09cf4f3c	2a6c7605

8 Cifra inversa

A decifração inverte as transformações e as aplica na ordem inversa: `InvShiftRows()` (deslocamento para a direita), `InvSubBytes()` (S-box inversa), `InvMixColumns()` (multiplicação por $a^{-1}(x)$) e `AddRoundKey()`, que é sua própria inversa.

```

InvCipher(byte in[4*Nb], byte out[4*Nb], word w[Nb*(Nr+1)])
begin
  state = in
  AddRoundKey(state, w[Nr*Nb, (Nr+1)*Nb-1])
  for round = Nr-1 step -1 downto 1
    InvShiftRows(state)
    InvSubBytes(state)
    AddRoundKey(state, w[round*Nb, (round+1)*Nb-1])
    InvMixColumns(state)
  end for
  InvShiftRows(state)
  InvSubBytes(state)

```

```
AddRoundKey(state, w[0, Nb-1])
  out = state
end
```

Cifra inversa equivalente. Como `SubBytes()`/`ShiftRows()` comutam e `InvMixColumns()` é linear em relação à entrada, é possível decifrar com a mesma sequência de transformações da cifra (cada uma substituída pela inversa), desde que as chaves de rodada das rodadas 1 a $N_r - 1$ passem por `InvMixColumns()` (vetor $dw[]$). Isso torna a estrutura mais eficiente.

9 Vetores de teste e implementação

Texto claro comum {00112233445566778899aabbccddeeff} e chaves {000102...} (Apêndice C):

Tabela 4: Vetores de exemplo do Apêndice C.

Versão	Chave (hex)	Texto cifrado (hex)
AES-128	000102030405060708090a0b0c0d0e0f	69c4e0d86a7b0430d8cdb78070b4c55a
AES-192	000102...14151617 (24 bytes)	dda97ca4864cdf06eaf70a0ec0d7191
AES-256	000102...1c1d1e1f (32 bytes)	8ea2b7ca516745bfeafc49904b496089

- Qualquer implementação que produza a mesma saída que o algoritmo do padrão é aceitável; implementações validadas por laboratório acreditado são consideradas em conformidade.
- Uma implementação deve suportar pelo menos um dos três tamanhos de chave.
- Segurança depende de mais que o algoritmo: gestão de chaves e modo de operação (por exemplo, NIST SP 800-21) também importam.
- Exportação de módulos criptográficos pode estar sujeita a controles governamentais.

Parte II – Protocolo SPI

10 Visão geral do SPI

O *Serial Peripheral Interface* é uma das interfaces mais usadas entre microcontroladores e CIs periféricos (sensores, ADCs, DACs, registradores de deslocamento, SRAM etc.). É uma interface **síncrona, full-duplex e mestre–escravo**: os dados do mestre ou do escravo são sincronizados na borda de subida ou de descida do clock, e ambos podem transmitir ao mesmo tempo. Pode ter 3 ou 4 fios; o resumo trata do SPI de **4 fios**.

11 Sinais e conexão

O dispositivo que gera o clock é o **mestre** (há apenas um mestre, com um ou vários escravos). Os quatro sinais são:

Tabela 5: Sinais do SPI de 4 fios.

Sinal	Função
SCLK (SPI CLK)	Clock gerado pelo mestre; sincroniza a transferência.
CS	<i>Chip select</i> , normalmente ativo em nível baixo; o mestre coloca 0 para selecionar o escravo e 1 para desconectá-lo do barramento.
MOSI	<i>Master Out, Slave In</i> : dados do mestre para o escravo.
MISO	<i>Master In, Slave Out</i> : dados do escravo para o mestre.

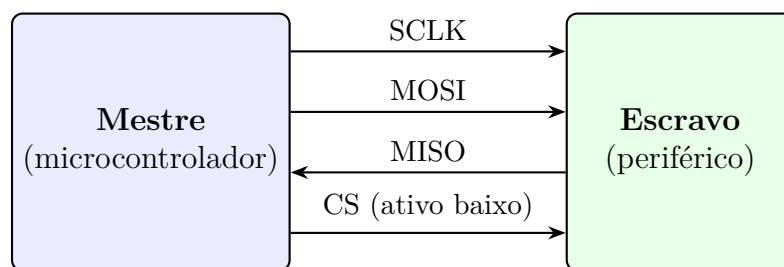


Figura 6: Conexão SPI entre um mestre e um escravo.

12 Transmissão de dados

Para iniciar a comunicação, o mestre gera o clock e coloca **CS em nível 0**. Durante a transferência, os bits são *deslocados* (enviados) e *amostrados* (lidos) ao mesmo tempo: a borda do clock sincroniza as duas ações. O número de bits por transferência e a frequência máxima do clock dependem do dispositivo (consultar o *data sheet*); o SPI suporta frequências de clock bem maiores que as do I²C.

13 Polaridade e fase do clock (CPOL e CPHA)

O mestre deve escolher a polaridade e a fase de acordo com o escravo:

- **CPOL**: nível do clock em repouso (*idle*): 0 = baixo, 1 = alto;
- **CPHA**: define em qual borda do clock os dados são amostrados e em qual são deslocados.

Combinando os dois bits obtém-se quatro modos.

Tabela 6: Modos SPI.

Modo	CPOL	CPHA	Clock em repouso	Amostragem / deslocamento
0	0	0	baixo	sobe: amostra; desce: desloca
1	0	1	baixo	desce: amostra; sobe: desloca
2	1	0	alto	desce: amostra; sobe: desloca
3	1	1	alto	sobe: amostra; desce: desloca

Observação: a numeração acima é a convencional (modo = $2 \cdot \text{CPOL} + \text{CPHA}$). O artigo da Analog Devices rotula os modos 2 e 3 com os valores de CPHA trocados, mas o comportamento descrito (nível em repouso e bordas) é o mesmo. Para uma comunicação correta, sempre confira o *data sheet* do componente.

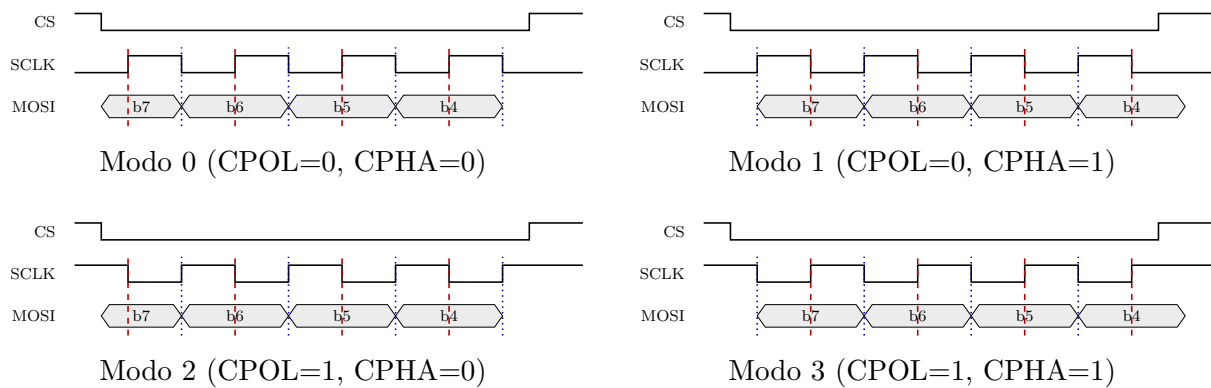


Figura 7: Diagramas de tempo ilustrativos dos quatro modos SPI. Linhas tracejadas vermelhas: bordas de amostragem; linhas pontilhadas azuis: bordas de deslocamento.

14 Configuração com vários escravos

Vários escravos podem compartilhar um único mestre, em modo regular ou em *daisy-chain*.

14.1 Modo regular

Cada escravo precisa de um **CS individual**; SCLK, MOSI e MISO são compartilhados. Se dois CS ficarem ativos ao mesmo tempo, os dados em MISO são corrompidos, pois o mestre não sabe qual escravo está transmitindo. A desvantagem é que o número de linhas de CS cresce com o número de escravos, consumindo GPIOs do mestre (uma solução é usar um mux para gerar os CS).

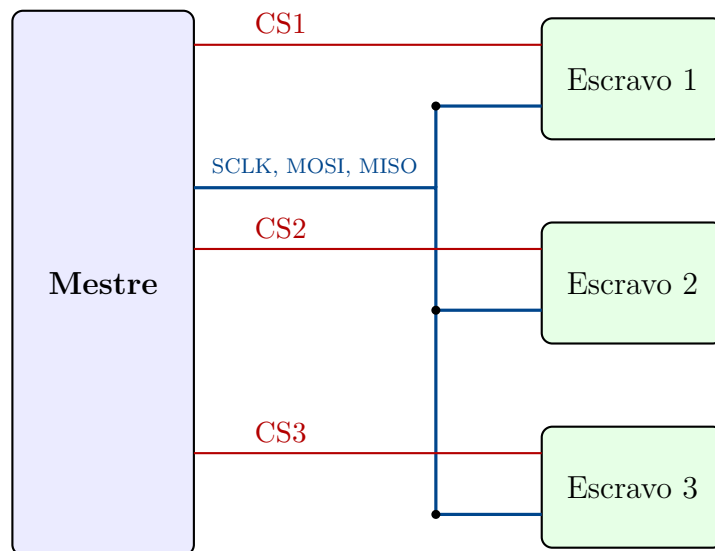


Figura 8: Múltiplos escravos em modo regular: barramento compartilhado (azul) e um CS por escravo (vermelho).

14.2 Modo *daisy-chain*

Os CS de todos os escravos são ligados juntos e todos recebem o mesmo clock. O MOSI do mestre vai ao primeiro escravo, a saída de cada escravo alimenta a entrada do seguinte e a saída do último volta ao MISO do mestre. Assim, o mestre usa **apenas 4 GPIOs, qualquer que seja o número de escravos**. Em contrapartida, o número de pulsos de clock cresce com a posição do escravo: em um sistema de 8 bits são necessários **24 pulsos** para o dado chegar ao 3.º escravo, contra apenas 8 no modo regular. Nem todo dispositivo suporta *daisy-chain*.

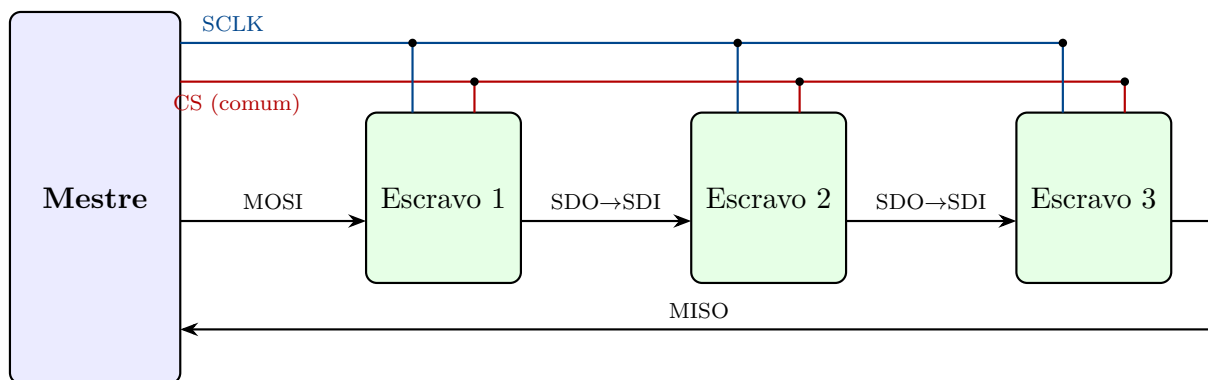


Figura 9: Daisy-chain: SCLK e CS comuns; os dados percorrem a cadeia de escravo em escravo.

15 Estudo de caso: chaves e muxes com interface SPI

Em sistemas com muitas chaves analógicas (por exemplo, instrumentação de teste), controlar cada chave por GPIO consome muitos pinos. O artigo usa como exemplo o

ADG1412, uma chave quádrupla SPST que exige 4 GPIOs. Numa matriz de cruzamento 4×4 com quatro ADG1412 são necessários **16 GPIOs**.

Alternativas:

- **Conversor serial–paralelo:** configurado por SPI e com saídas paralelas para as chaves; reduz GPIOs, mas aumenta a lista de materiais (mais um componente).
- **Chaves com interface SPI:** dispensam o conversor; com CS individuais bastam **7 GPIOs** para as quatro chaves (SCLK, MOSI, MISO e 4 CS).
- **SPI em *daisy-chain*:** apenas **4 GPIOs**, independentemente do número de chaves.

No exemplo de uma matriz de cruzamento 4×8 com oito chaves quádruplas SPST em placa de 6 camadas, as chaves com SPI proporcionam cerca de 20% de redução de área de placa.

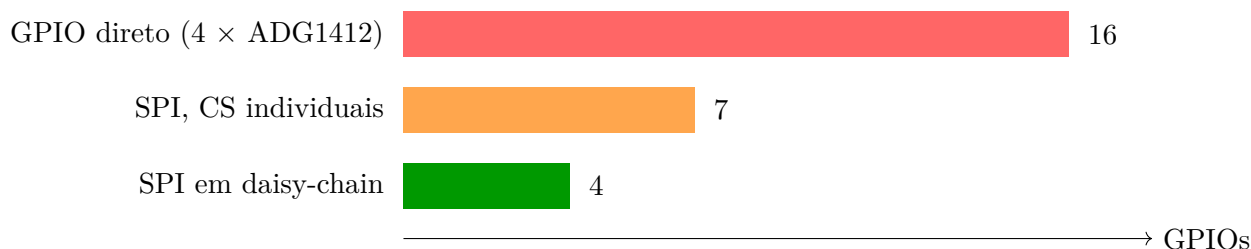


Figura 10: GPIOs do microcontrolador necessários para controlar quatro chaves quádruplas.

16 Resumo comparativo

Tabela 7: Quadro-resumo do SPI.

Tipo	Síncrono, full-duplex, mestre–escravo
Fios	3 ou 4 (foco: SCLK, CS, MOSI, MISO)
Mestres	Apenas um
Escravos	Um ou vários (CS individual ou daisy-chain)
Modos	4 (combinações de CPOL e CPHA)
Velocidade	Clock bem mais alto que o do I ² C (ver <i>data sheet</i>)

Referências

- [1] NIST. *Advanced Encryption Standard (AES)*. FIPS PUB 197, 26 nov. 2001; atualizado em 9 mai. 2023 (FIPS 197-upd1). <https://doi.org/10.6028/NIST.FIPS.197-upd1>.
- [2] DAEMEN, J.; RIJMEN, V. *AES Proposal: Rijndael*. AES Algorithm Submission, 1999.
- [3] DHAKER, P. *Introduction to SPI Interface*. Analog Dialogue 52-09, Analog Devices, set. 2018.
- [4] USACH, M. *AN-1248 Application Note: SPI Interface*. Analog Devices, 2015.
- [5] NUGENT, S. *Precision SPI Switch Configuration Increases Channel Density*. Analog Dialogue, mai. 2017.